Estruturas de Decisão e Repetição em COBOL

Objetivos da Aula
Ao final desta aula o aluno deverá ser capaz de:

- Utilizar estruturas de decisão com `IF` e `ELSE`
- Trabalhar com operadores relacionais e lógicos
- Utilizar `IF` aninhados para múltiplas condições
- Utilizar `EVALUATE` como alternativa ao `IF` múltiplo
- Criar menus interativos com `EVALUATE`
- Modularizar programas com `PERFORM`
- Criar laços com `PERFORM UNTIL`
- Utilizar `PERFORM VARYING` para loops contados
- Utilizar `PERFORM TIMES` para repetições fixas
- Desenvolver programas completos e interativos

Revisão da Aula Anterior (10 min)

O que já sabemos fazer:

Operação	Comando	Exemplo
Copiar dados	MOVE	MOVE A TO B
Somar	ADD	ADD A TO B
Subtrair	SUBTRACT	SUBTRACT A FROM B
Multiplicar	MULTIPLY	MULTIPLY A BY B
Dividir	DIVIDE	DIVIDE A BY B

Problema atual:

```
ACCEPT WS-NOTA.     ← Recebeu 7
???                 ← Como decidir se foi aprovado?
```

> 🤔 ***Desafio de hoje:*** *Fazer o programa tomar decisões e repetir tarefas!*

1 – Operadores Relacionais (10 min)

O que são?

Operadores que **comparam valores** e retornam Verdadeiro ou Falso.

Analogia: Balança de Comparação

```
10 > 5  → VERDADEIRO
10 < 5  → FALSO
10 = 10 → VERDADEIRO
```

Tabela de Operadores

Operador	Significado	Exemplo (Verdadeiro)
`=`	Igual	`10 = 10`
`>`	Maior	`10 > 5`
`<`	Menor	`5 < 10`
`>=`	Maior ou igual	`10 >= 10`
`<=`	Menor ou igual	`5 <= 10`
`NOT =`	Diferente	`10 NOT = 5`
`>`	Maior (também pode ser `GREATER THAN`)	
`<`	Menor (também pode ser `LESS THAN`)	

Exemplos Práticos

```
*>> Verificando idade
IF WS-IDADE >= 18
    DISPLAY "MAIOR DE IDADE"
END-IF.

*>> Verificando nota
IF WS-NOTA = 10
    DISPLAY "NOTA MÁXIMA!"
END-IF.

*>> Verificando diferente
IF WS-CODIGO NOT = "A001"
    DISPLAY "CÓDIGO INVÁLIDO"
END-IF.
```

⚠ **Atenção à Sintaxe**

```
*>> FORMA CORRETA
IF WS-IDADE >= 18

*>> FORMA INCORRETA (ESPAÇO EM BRANCO)
IF WS-IDADE > = 18  ← NÃO FUNCIONA!
```

> 💡 ***Dica:*** *Sempre use os operadores sem espaços entre os caracteres.*

2 – IF Simples (15 min)

O que é IF?

Executa um bloco de código **se a condição for verdadeira**.

Analogia: Porta com Fechadura

```
SE a chave for correta (condição verdadeira)
    ENTÃO abra a porta (executa comando)
```

Sintaxe

```
IF condição
    comando(s)
END-IF.
```

Exemplo Simples

```
IDENTIFICATION DIVISION.
PROGRAM-ID. IF-SIMPLES.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOTA PIC 9(3).

PROCEDURE DIVISION.
    DISPLAY "DIGITE A NOTA:".
    ACCEPT WS-NOTA.

    IF WS-NOTA >= 60
        DISPLAY "APROVADO!"
    END-IF.

    STOP RUN.
```

Execução 1:

```
DIGITE A NOTA:
75
APROVADO!
```

Execução 2:

```
DIGITE A NOTA:
45
                  ← Não mostra nada!
```

IF com Múltiplos Comandos

```
IF WS-NOTA >= 60
    DISPLAY "=========================="
    DISPLAY "PARABÉNS! APROVADO!"
    DISPLAY "=========================="
END-IF.
```

IF com Nível 88

```
01 WS-IDADE PIC 9(3).
   88 MAIOR-IDADE VALUE 18 THRU 150.

IF MAIOR-IDADE
    DISPLAY "ACESSO PERMITIDO"
END-IF.
```

> 💡 ***Dica**: Usar nível 88 torna o código mais legível!*

Exercício de Fixação

Crie um programa que:

1. Receba a idade do usuário
2. Se for maior que 60, exiba "VOCÊ TEM DIREITO A VAGAS PREFERENCIAIS"

Resposta:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. IDOSO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-IDADE PIC 9(3).

PROCEDURE DIVISION.
    DISPLAY "DIGITE SUA IDADE:".
    ACCEPT WS-IDADE.

    IF WS-IDADE > 60
        DISPLAY "VOCÊ TEM DIREITO A VAGAS PREFERENCIAIS"
    END-IF.

    STOP RUN.
```

3 – IF ... ELSE (15 min)

O que é IF...ELSE?

Executa um bloco **se a condição for verdadeira** e outro **se for falsa**.

Analogia: Semáforo

```
SE o sinal estiver VERDE (condição verdadeira)
    ENTÃO atravesse a rua (bloco verdadeiro)
SENÃO
    ESPERE o sinal abrir (bloco falso)
```

Sintaxe

```
IF condição
    comando(s) VERDADEIRO
ELSE
    comando(s) FALSO
END-IF.
```

Exemplo - Aprovação

```
IDENTIFICATION DIVISION.
PROGRAM-ID. IF-ELSE.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOTA PIC 9(3).

PROCEDURE DIVISION.
    DISPLAY "DIGITE A NOTA:".
    ACCEPT WS-NOTA.

    IF WS-NOTA >= 60
        DISPLAY "APROVADO!"
    ELSE
        DISPLAY "REPROVADO!"
    END-IF.

    STOP RUN.
```

Execução:

```
DIGITE A NOTA:
75
APROVADO!

DIGITE A NOTA:
45
REPROVADO!
```

Exemplo - Par ou Ímpar

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PAR-IMPAR.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NUMERO PIC 9(5).
01 WS-RESTO  PIC 9(5).

PROCEDURE DIVISION.
    DISPLAY "DIGITE UM NÚMERO:".
    ACCEPT WS-NUMERO.

    DIVIDE WS-NUMERO BY 2
        GIVING WS-RESTO
        REMAINDER WS-RESTO.
```

```
    IF WS-RESTO = 0
        DISPLAY WS-NUMERO " É PAR!"
    ELSE
        DISPLAY WS-NUMERO " É ÍMPAR!"
    END-IF.

    STOP RUN.
```

IF com Múltiplas Linhas

```
IF WS-SALDO >= 0
    DISPLAY "=========================="
    DISPLAY "SALDO POSITIVO"
    DISPLAY "VALOR: R$ " WS-SALDO
    DISPLAY "=========================="
ELSE
    DISPLAY "=========================="
    DISPLAY "SALDO NEGATIVO"
    DISPLAY "VALOR: R$ " WS-SALDO
    DISPLAY "=========================="
END-IF.
```

Exercício de Fixação

Crie um programa que:

1. Receba um número
2. Se for positivo, exiba "NÚMERO POSITIVO"
3. Se for negativo, exiba "NÚMERO NEGATIVO"
4. Se for zero, exiba "NÚMERO NEUTRO"

Dica: Use `IF ... ELSE` aninhado (próximo tópico)!

4 – IF Aninhados (15 min)

O que são?

Quando colocamos um `IF` **dentro** de outro `IF`.

Analogia: Portaria de Condomínio

```
SE é morador (condição externa verdadeira)
    SE tem crachá (condição interna verdadeira)
        ENTÃO pode entrar
    SENÃO
        precisa se identificar
SENÃO
    não pode entrar
```

Sintaxe

```
IF condição1
    IF condição2
        comando(s)
    ELSE
        comando(s)
    END-IF
ELSE
    comando(s)
END-IF.
```

Exemplo - Classificação de Notas

```
IDENTIFICATION DIVISION.
PROGRAM-ID. IF-ANINHADO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOTA PIC 9(3).

PROCEDURE DIVISION.
    DISPLAY "DIGITE A NOTA:".
    ACCEPT WS-NOTA.

    IF WS-NOTA >= 60
        IF WS-NOTA >= 90
            DISPLAY "APROVADO COM LOUVOR!"
        ELSE
            DISPLAY "APROVADO!"
        END-IF
    ELSE
```

```
        IF WS-NOTA >= 40
            DISPLAY "RECUPERAÇÃO"
        ELSE
            DISPLAY "REPROVADO!"
        END-IF
    END-IF.

    STOP RUN.
```

Execução:

```
DIGITE A NOTA:
95
APROVADO COM LOUVOR!

DIGITE A NOTA:
75
APROVADO!

DIGITE A NOTA:
50
RECUPERAÇÃO

DIGITE A NOTA:
35
REPROVADO!
```

Exemplo - Validação Completa

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VALIDA-LOGIN.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-USUARIO PIC X(10).
01 WS-SENHA   PIC X(10).

PROCEDURE DIVISION.
    DISPLAY "USUÁRIO:".
    ACCEPT WS-USUARIO.
    DISPLAY "SENHA:".
    ACCEPT WS-SENHA.

    IF WS-USUARIO = "ADMIN"
        IF WS-SENHA = "1234"
            DISPLAY "ACESSO LIBERADO!"
        ELSE
            DISPLAY "SENHA INCORRETA!"
        END-IF
    ELSE
        DISPLAY "USUÁRIO NÃO ENCONTRADO!"
    END-IF.

    STOP RUN.
```

⚠ Cuidados com IF Aninhados

Problema	Solução
Muitos níveis (código confuso)	Usar `EVALUATE`
Esquecer o `END-IF` correspondente	Contar os `IF` abertos
Colocar `ELSE` no lugar errado	Usar indentação clara

> 💡 ***Dica**: Cada `IF` deve ter seu `END-IF` correspondente.*

5 – Operadores Lógicos (15 min)

Para que servem?

Combinam **duas ou mais condições** em uma única expressão.

Analogia: Regras de uma Festa

```
Pode entrar SE (tem convite) E (é maior de idade)
Pode estacionar SE (é cliente) OU (tem autorização)
```

AND - Todas as condições devem ser verdadeiras

```
IF WS-IDADE >= 18 AND WS-IDADE <= 65
    DISPLAY "FAIXA ETÁRIA ADULTA"
END-IF.
```

Tabela Verdade AND:

Condição 1	Condição 2	Resultado
V	V	V
V	F	F
F	V	F
F	F	F

OR - Pelo menos uma condição deve ser verdadeira

```
IF WS-CIDADE = "SAO PAULO" OR WS-CIDADE = "BARUERI"
    DISPLAY "REGIAO METROPOLITANA"
END-IF.
```

Tabela Verdade OR:

Condição 1	Condição 2	Resultado
V	V	V
V	F	V
F	V	V
F	F	F

NOT - Inverte o resultado

```
IF NOT WS-ATIVO
    DISPLAY "USUÁRIO INATIVO"
END-IF.
```

Tabela Verdade NOT:

Condição	Resultado
V	F
F	V

Exemplo Completo - Sistema de Empréstimo

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EMPRESTIMO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-IDADE    PIC 9(3).
01 WS-RENDA    PIC 9(6)V99.
01 WS-SCORE    PIC 9(3).
01 WS-RESULT   PIC X(20).

PROCEDURE DIVISION.
    DISPLAY "=== ANÁLISE DE EMPRÉSTIMO ===".
    DISPLAY "IDADE:".
    ACCEPT WS-IDADE.
    DISPLAY "RENDA MENSAL:".
    ACCEPT WS-RENDA.
    DISPLAY "SCORE DE CRÉDITO:".
    ACCEPT WS-SCORE.

    IF (WS-IDADE >= 18)
       AND (WS-RENDA > 2000)
       AND (WS-SCORE >= 600)
       IF WS-RENDA > 5000
           MOVE "APROVADO - LIMITE ALTO" TO WS-RESULT
       ELSE
           MOVE "APROVADO - LIMITE BAIXO" TO WS-RESULT
       END-IF
    ELSE
       MOVE "REPROVADO - CRITÉRIOS NÃO ATENDIDOS" TO WS-RESULT
    END-IF.

    DISPLAY "======================".
    DISPLAY "RESULTADO: " WS-RESULT.
    DISPLAY "======================".
    STOP RUN.
```

Combinações Complexas

```
IF (WS-IDADE >= 18 AND WS-IDADE <= 65)
   OR (WS-IDADE > 65 AND WS-APOSENTADO = "S")
    DISPLAY "PODE PARTICIPAR DO SISTEMA"
END-IF.
```

6 – EVALUATE (20 min)

O que é EVALUATE?

Uma alternativa mais **limpa e organizada** ao `IF` múltiplo.

Analogia: Cardápio de Restaurante

```
ESCOLHA o prato (EVALUATE variável)
    CASO 1: Pizza (bloco 1)
    CASO 2: Hambúrguer (bloco 2)
    CASO 3: Salada (bloco 3)
    CASO CONTRÁRIO: Opção inválida
FIM
```

Sintaxe Básica

```
EVALUATE variável
    WHEN valor1
        comando(s)
    WHEN valor2
        comando(s)
    WHEN valor3
        comando(s)
    WHEN OTHER
        comando(s)
END-EVALUATE.
```

Exemplo - Menu

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EVALUATE-MENU.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-OPCAO PIC 9.

PROCEDURE DIVISION.
    DISPLAY "MENU PRINCIPAL".
    DISPLAY "=============".
    DISPLAY "1 - CADASTRAR".
    DISPLAY "2 - CONSULTAR".
    DISPLAY "3 - EXCLUIR".
    DISPLAY "4 - SAIR".
    DISPLAY "OPÇÃO:".
    ACCEPT WS-OPCAO.

    EVALUATE WS-OPCAO
```

```
        WHEN 1
            DISPLAY "CADASTRO SELECIONADO"
        WHEN 2
            DISPLAY "CONSULTA SELECIONADA"
        WHEN 3
            DISPLAY "EXCLUSÃO SELECIONADA"
        WHEN 4
            DISPLAY "SAINDO..."
        WHEN OTHER
            DISPLAY "OPÇÃO INVÁLIDA!"
    END-EVALUATE.

    STOP RUN.
```

EVALUATE com Intervalos

```
IDENTIFICATION DIVISION.
PROGRAM-ID. EVALUATE-NOTA.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOTA PIC 9(3).

PROCEDURE DIVISION.
    DISPLAY "DIGITE A NOTA:".
    ACCEPT WS-NOTA.

    EVALUATE TRUE
        WHEN WS-NOTA >= 90
            DISPLAY "CONCEITO A"
        WHEN WS-NOTA >= 80
            DISPLAY "CONCEITO B"
        WHEN WS-NOTA >= 70
            DISPLAY "CONCEITO C"
        WHEN WS-NOTA >= 60
            DISPLAY "CONCEITO D"
        WHEN OTHER
            DISPLAY "CONCEITO F"
    END-EVALUATE.

    STOP RUN.
```

EVALUATE com Múltiplos Valores

```
EVALUATE WS-MES
    WHEN 1
    WHEN 2
    WHEN 12
        DISPLAY "FÉRIAS ESCOLARES"
    WHEN 3
    WHEN 4
    WHEN 5
    WHEN 6
        DISPLAY "PERÍODO LETIVO"
    WHEN OTHER
        DISPLAY "OUTRO PERÍODO"
END-EVALUATE.
```

EVALUATE vs IF - Comparação

Com IF (mais complexo):

```
IF WS-OPCAO = 1
    PERFORM CADASTRAR
ELSE
    IF WS-OPCAO = 2
        PERFORM CONSULTAR
    ELSE
        IF WS-OPCAO = 3
            PERFORM EXCLUIR
        ELSE
            DISPLAY "OPÇÃO INVÁLIDA"
        END-IF
    END-IF
END-IF.
```

Com EVALUATE (mais limpo):

```
EVALUATE WS-OPCAO
    WHEN 1
        PERFORM CADASTRAR
    WHEN 2
        PERFORM CONSULTAR
    WHEN 3
        PERFORM EXCLUIR
    WHEN OTHER
        DISPLAY "OPÇÃO INVÁLIDA"
END-EVALUATE.
```

> 💡 ***Vantagem**: Código mais legível e organizado!*

7 – PERFORM Simples (10 min)

O que é PERFORM?

Permite **chamar** um bloco de código (parágrafo) de dentro da PROCEDURE DIVISION.

Analogia: Receita de Bolo

```
Pegue os ingredientes (PARÁGRAFO 1)
PERFORM Preparar massa (chama PARÁGRAFO 2)
PERFORM Assar bolo (chama PARÁGRAFO 3)
PERFORM Decorar bolo (chama PARÁGRAFO 4)
```

Sintaxe

```
PROCEDURE DIVISION.
    PERFORM NOME-DO-PARAGRAFO.
    PERFORM OUTRO-PARAGRAFO.

NOME-DO-PARAGRAFO.
    comando(s)
    EXIT.

OUTRO-PARAGRAFO.
    comando(s)
    EXIT.
```

Exemplo - Programa Modular

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PERFORM-EXEMPLO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOME PIC X(30).

PROCEDURE DIVISION.
    PERFORM CABECALHO.
    PERFORM ENTRADA-DADOS.
    PERFORM EXIBIR-CADASTRO.
    PERFORM RODAPE.
    STOP RUN.

CABECALHO.
    DISPLAY "=========================".
    DISPLAY "SISTEMA DE CADASTRO".
    DISPLAY "=========================".
    EXIT.
```

```
ENTRADA-DADOS.
    DISPLAY "DIGITE O NOME:".
    ACCEPT WS-NOME.
    EXIT.

EXIBIR-CADASTRO.
    DISPLAY "CADASTRO REALIZADO".
    DISPLAY "NOME: " WS-NOME.
    EXIT.

RODAPE.
    DISPLAY "========================".
    DISPLAY "FIM DO PROGRAMA".
    DISPLAY "========================".
    EXIT.
```

Saída:

```
========================
SISTEMA DE CADASTRO
========================
DIGITE O NOME:
JOAO
CADASTRO REALIZADO
NOME: JOAO
========================
FIM DO PROGRAMA
========================
```

Vantagens do PERFORM

Benefício	Descrição
Organização	Código mais estruturado
Reutilização	Mesmo bloco usado várias vezes
Manutenção	Corrigir em um único lugar
Legibilidade	Fica claro o que o programa faz

8 – PERFORM UNTIL (15 min)

O que é PERFORM UNTIL?

Executa um bloco de código **repetidamente até que uma condição seja verdadeira**.

Analogia: Porta com Contador

```
Continue tentando a senha (executa bloco)
ATÉ a senha estar correta (condição verdadeira)
```

Sintaxe

```
PERFORM UNTIL condição
    comando(s)
END-PERFORM.
```

Exemplo - Validação de Idade

```
IDENTIFICATION DIVISION.
PROGRAM-ID. UNTIL-EXEMPLO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-IDADE PIC 9(3).

PROCEDURE DIVISION.
    PERFORM UNTIL WS-IDADE >= 18
        DISPLAY "DIGITE A IDADE (MÍNIMO 18):".
        ACCEPT WS-IDADE
    END-PERFORM.

    DISPLAY "IDADE VÁLIDA: " WS-IDADE.
    STOP RUN.
```

Execução:

```
DIGITE A IDADE (MÍNIMO 18):
15
DIGITE A IDADE (MÍNIMO 18):
12
DIGITE A IDADE (MÍNIMO 18):
20
IDADE VÁLIDA: 20
```

Exemplo - Menu com Saída

```
IDENTIFICATION DIVISION.
PROGRAM-ID. UNTIL-MENU.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-OPCAO PIC 9.

PROCEDURE DIVISION.
    PERFORM UNTIL WS-OPCAO = 5
        DISPLAY "MENU PRINCIPAL"
        DISPLAY "=============="
        DISPLAY "1 - CADASTRAR"
        DISPLAY "2 - CONSULTAR"
        DISPLAY "3 - ALTERAR"
        DISPLAY "4 - EXCLUIR"
        DISPLAY "5 - SAIR"
        DISPLAY "OPÇÃO:".
        ACCEPT WS-OPCAO

        EVALUATE WS-OPCAO
            WHEN 1
                DISPLAY "CADASTRANDO..."
            WHEN 2
                DISPLAY "CONSULTANDO..."
            WHEN 3
                DISPLAY "ALTERANDO..."
            WHEN 4
                DISPLAY "EXCLUINDO..."
            WHEN 5
                DISPLAY "SAINDO..."
            WHEN OTHER
                DISPLAY "OPÇÃO INVÁLIDA!"
        END-EVALUATE
    END-PERFORM.

    DISPLAY "FIM DO PROGRAMA".
    STOP RUN.
```

Exemplo - Loop com Contador

```
IDENTIFICATION DIVISION.
PROGRAM-ID. UNTIL-CONTADOR.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-CONTADOR PIC 99 VALUE 1.

PROCEDURE DIVISION.
    PERFORM UNTIL WS-CONTADOR > 10
        DISPLAY "CONTADOR: " WS-CONTADOR
        ADD 1 TO WS-CONTADOR
    END-PERFORM.

    DISPLAY "FIM DO LOOP".
    STOP RUN.
```

Saída:

```
CONTADOR: 1
CONTADOR: 2
CONTADOR: 3
CONTADOR: 4
CONTADOR: 5
CONTADOR: 6
CONTADOR: 7
CONTADOR: 8
CONTADOR: 9
CONTADOR: 10
FIM DO LOOP
```

9 – PERFORM TIMES (10 min)

O que é PERFORM TIMES?

Executa um bloco de código **um número fixo de vezes**.

Analogia: Máquina de Café

```
Aperte o botão 3 VEZES (executa 3 vezes)
Cada vez, uma xícara de café (bloco executado)
```

Sintaxe

```
PERFORM numero TIMES
    comando(s)
END-PERFORM.
```

Exemplo - Mensagem Repetida

```
IDENTIFICATION DIVISION.
PROGRAM-ID. TIMES-EXEMPLO.

PROCEDURE DIVISION.
    PERFORM 5 TIMES
        DISPLAY "APRENDENDO COBOL!"
    END-PERFORM.

    STOP RUN.
```

Saída:

```
APRENDENDO COBOL!
APRENDENDO COBOL!
APRENDENDO COBOL!
APRENDENDO COBOL!
APRENDENDO COBOL!
```

Exemplo - Tabuada (com VARIÁVEL)

```
IDENTIFICATION DIVISION.
PROGRAM-ID. TIMES-TABUADA.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NUMERO PIC 9(3).
01 WS-CONT   PIC 99.
01 WS-RESULT PIC 9(5).

PROCEDURE DIVISION.
```

```
    DISPLAY "DIGITE UM NÚMERO:".
    ACCEPT WS-NUMERO.

    MOVE 1 TO WS-CONT.
    PERFORM 10 TIMES
        MULTIPLY WS-NUMERO BY WS-CONT
            GIVING WS-RESULT
        DISPLAY WS-NUMERO " X " WS-CONT " = " WS-RESULT
        ADD 1 TO WS-CONT
    END-PERFORM.

    STOP RUN.
```

Saída:

```
DIGITE UM NÚMERO:
5
5 X 1 = 5
5 X 2 = 10
5 X 3 = 15
...
5 X 10 = 50
```

10 – PERFORM VARYING (15 min)

O que é PERFORM VARYING?

Um loop **mais poderoso** que controla automaticamente uma variável de contagem.

Analogia: Esteira Rolante

```
Para CADA item na esteira (VARYING posição)
    Processe o item (comando)
ATÉ não ter mais itens (condição)
```

Sintaxe

```
PERFORM VARYING contador
    FROM valor-inicial
    BY incremento
    UNTIL condição
    comando(s)
END-PERFORM.
```

Exemplo - Números de 1 a 10

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VARYING-EXEMPLO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I PIC 99.

PROCEDURE DIVISION.
    PERFORM VARYING WS-I
        FROM 1 BY 1
        UNTIL WS-I > 10
        DISPLAY WS-I
    END-PERFORM.

    STOP RUN.
```

Saída:

```
1
2
3
4
5
6
7
8
9
10
```

Exemplo - Contagem Regressiva

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VARYING-REG.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I PIC 99.

PROCEDURE DIVISION.
    PERFORM VARYING WS-I
        FROM 10 BY -1
        UNTIL WS-I < 1
        DISPLAY WS-I
    END-PERFORM.

    DISPLAY "FOGO!".
    STOP RUN.
```

Saída:

```
10
9
8
7
6
5
4
3
2
1
FOGO!
```

Exemplo - Soma de Números

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VARYING-SOMA.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-I PIC 99.
01 WS-SOMA PIC 999.

PROCEDURE DIVISION.
    MOVE 0 TO WS-SOMA.

    PERFORM VARYING WS-I
        FROM 1 BY 1
        UNTIL WS-I > 100
        ADD WS-I TO WS-SOMA
    END-PERFORM.

    DISPLAY "SOMA DE 1 A 100: " WS-SOMA.
    STOP RUN.
```

Saída:

```
SOMA DE 1 A 100: 5050
```

Exemplo - Vetor com PERFORM VARYING

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VETOR-VARYING.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NOMES.
   05 WS-NOME PIC X(20) OCCURS 5 TIMES.
01 WS-I PIC 99.

PROCEDURE DIVISION.
    *> PREENCHENDO
    MOVE "JOAO"   TO WS-NOME(1)
    MOVE "MARIA"  TO WS-NOME(2)
    MOVE "PEDRO"  TO WS-NOME(3)
    MOVE "ANA"    TO WS-NOME(4)
    MOVE "CARLOS" TO WS-NOME(5)

    *> EXIBINDO
    PERFORM VARYING WS-I
        FROM 1 BY 1
        UNTIL WS-I > 5
        DISPLAY WS-I ". " WS-NOME(WS-I)
    END-PERFORM.

    STOP RUN.
```

Saída:

```
1. JOAO
2. MARIA
3. PEDRO
4. ANA
5. CARLOS
```

Exercício Guiado (10 min)

Programa - Contagem de Pares

Crie um programa que exiba os números pares de 1 a 20.

Resolução:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PARES.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-NUMERO PIC 99.

PROCEDURE DIVISION.
    PERFORM VARYING WS-NUMERO
        FROM 2 BY 2
        UNTIL WS-NUMERO > 20
        DISPLAY WS-NUMERO
    END-PERFORM.

    STOP RUN.
```

Saída:

```
2
4
6
8
10
12
14
16
18
20
```

Atividade Prática da Aula (30 minutos)

🎯 Sistema de Calculadora Completa

Objetivo:
Criar uma calculadora que funcione em loop até o usuário escolher sair.

Programa Completo

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. CALCULADORA-COMPLETA.

       DATA DIVISION.
       WORKING-STORAGE SECTION.
       01 WS-OPCAO PIC 9.
       01 WS-NUM1  PIC 9(5)V99.
       01 WS-NUM2  PIC 9(5)V99.
       01 WS-RESULT PIC 9(7)V99.
       01 WS-CONT  PIC 99.
       01 WS-HIST  PIC X(40) OCCURS 10 TIMES.

       PROCEDURE DIVISION.
           PERFORM UNTIL WS-OPCAO = 6
               PERFORM EXIBIR-MENU
               ACCEPT WS-OPCAO

               IF WS-OPCAO = 6
                   DISPLAY "SAINDO..."
               ELSE
                   PERFORM LER-NUMEROS
                   PERFORM CALCULAR
                   PERFORM EXIBIR-RESULTADO
               END-IF
           END-PERFORM.

           DISPLAY "CALCULADORA ENCERRADA".
           STOP RUN.

       EXIBIR-MENU.
           DISPLAY "==================================".
           DISPLAY "         CALCULADORA COBOL".
           DISPLAY "==================================".
           DISPLAY "1 - SOMAR".
           DISPLAY "2 - SUBTRAIR".
           DISPLAY "3 - MULTIPLICAR".
           DISPLAY "4 - DIVIDIR".
```

```
           DISPLAY "5 - LIMPAR TELA (SIMULADO)".
           DISPLAY "6 - SAIR".
           DISPLAY "==================================".
           DISPLAY "OPÇÃO: ".
           EXIT.

       LER-NUMEROS.
           DISPLAY "DIGITE O PRIMEIRO NÚMERO:".
           ACCEPT WS-NUM1.
           DISPLAY "DIGITE O SEGUNDO NÚMERO:".
           ACCEPT WS-NUM2.
           EXIT.

       CALCULAR.
           EVALUATE WS-OPCAO
               WHEN 1
                   ADD WS-NUM1 WS-NUM2
                       GIVING WS-RESULT
               WHEN 2
                   SUBTRACT WS-NUM2
                       FROM WS-NUM1
                       GIVING WS-RESULT
               WHEN 3
                   MULTIPLY WS-NUM1 BY WS-NUM2
                       GIVING WS-RESULT
               WHEN 4
                   IF WS-NUM2 = 0
                       MOVE 0 TO WS-RESULT
                       DISPLAY "ERRO! DIVISÃO POR ZERO!"
                   ELSE
                       DIVIDE WS-NUM1 BY WS-NUM2
                           GIVING WS-RESULT ROUNDED
                   END-IF
               WHEN 5
                   PERFORM 3 TIMES
                       DISPLAY " "
                   END-PERFORM
               WHEN OTHER
                   DISPLAY "OPÇÃO INVÁLIDA!"
           END-EVALUATE.
           EXIT.

       EXIBIR-RESULTADO.
           IF WS-OPCAO NOT = 4 OR WS-NUM2 NOT = 0
               DISPLAY "==================================".
               DISPLAY "RESULTADO: " WS-RESULT.
```

```
               DISPLAY "====================================".
           END-IF.

           DISPLAY " ".
           DISPLAY "Pressione ENTER para continuar...".
           ACCEPT WS-CONT.
           EXIT.
```

Saída Esperada:

```
==================================
         CALCULADORA COBOL
==================================
1 - SOMAR
2 - SUBTRAIR
3 - MULTIPLICAR
4 - DIVIDIR
5 - LIMPAR TELA (SIMULADO)
6 - SAIR
==================================
OPÇÃO:
1

DIGITE O PRIMEIRO NÚMERO:
10
DIGITE O SEGUNDO NÚMERO:
20
==================================
RESULTADO: 30.00
==================================

Pressione ENTER para continuar...

==================================
         CALCULADORA COBOL
==================================
1 - SOMAR
2 - SUBTRAIR
3 - MULTIPLICAR
4 - DIVIDIR
5 - LIMPAR TELA (SIMULADO)
6 - SAIR
==================================
OPÇÃO:
4
```

```
DIGITE O PRIMEIRO NÚMERO:
15
DIGITE O SEGUNDO NÚMERO:
3
==================================
RESULTADO: 5.00
==================================

Pressione ENTER para continuar...
```

Desafio Extra (Opcional)

🏆 **Sistema de Login com 3 Tentativas**

Criar um sistema onde o usuário tem 3 tentativas para acertar usuário e senha.

Entrada:

```
USUÁRIO: ADMIN
SENHA: 1234
```

Regras:
- 3 tentativas
- A cada erro, mostra "TENTATIVA X DE 3"
- Após 3 erros, bloqueia o sistema
- Se acertar, libera acesso

Resolução:

```
IDENTIFICATION DIVISION.
PROGRAM-ID. LOGIN-SEGURO.

DATA DIVISION.
WORKING-STORAGE SECTION.
01 WS-USUARIO PIC X(10).
01 WS-SENHA   PIC X(10).
01 WS-TENTATIVAS PIC 9.
01 WS-BLOQUEADO PIC X(1) VALUE "N".
   88 BLOQUEADO VALUE "S".

PROCEDURE DIVISION.
    MOVE 0 TO WS-TENTATIVAS.

    PERFORM UNTIL WS-TENTATIVAS >= 3 OR WS-USUARIO = "ADMIN"
        ADD 1 TO WS-TENTATIVAS

        DISPLAY "TENTATIVA " WS-TENTATIVAS " DE 3".
```

```
        DISPLAY "USUÁRIO:".
        ACCEPT WS-USUARIO.
        DISPLAY "SENHA:".
        ACCEPT WS-SENHA.

        IF WS-USUARIO = "ADMIN" AND WS-SENHA = "1234"
            DISPLAY "ACESSO LIBERADO!"
        ELSE
            DISPLAY "ACESSO NEGADO!"
        END-IF
    END-PERFORM.

    IF WS-TENTATIVAS >= 3
        DISPLAY "=========================".
        DISPLAY "SISTEMA BLOQUEADO!"
        DISPLAY "=========================".
    END-IF.

    STOP RUN.
```